

CP468 Artificial Intelligence

Assignment 2: Connect Four AI — Report

Team Members: Asad Malik, Saad Siddiqui, Adam Ali, Daniyal Naqvi, Omar Hamza

Date: [Submission Date]

1. Introduction

This report describes the design, implementation, and evaluation of three intelligent agents for the game of Connect Four, developed for CP468 Assignment 2. Connect Four is a two-player, zero-sum, perfect-information adversarial game played on a 7-column by 6-row board, making it a natural fit for classical adversarial search techniques covered in this course.

The goal of the assignment was not to build a polished game application, but to correctly implement and empirically compare three agents of increasing sophistication: a random baseline, a hand-crafted rule-based agent, and a Minimax search agent with a windowed heuristic evaluation function. All three agents share a single game engine, ensuring the rules of the game are enforced identically regardless of which agent is playing.

Section 2 describes the system architecture. Section 3 details each agent's decision-making logic, including the mandatory tie-breaking rule and, for the Minimax agent, the heuristic evaluation function. Section 4 presents the experimental protocol and results across all three required pairings. Section 5 discusses the results and proposes techniques that could further improve play. Section 6 lists team contributions.

2. System Design

The implementation is split into four Python modules, each with a single responsibility, so that all three agents interact with the game through one shared, tested interface rather than each re-implementing board logic.

2.1 Game Engine (engine.py)

The engine represents the board as a 6x7 list of lists, where row 0 is the bottom row and row 5 is the top row. Cell values are 0 (empty), 1 (player 1), or 2 (player 2). The engine exposes a small, pure-functional interface:

- `legal_moves(board)` — returns the list of non-full columns
- `apply_move(board, col, player)` — returns a new board with the disc dropped into the lowest empty cell of the given column (gravity), without mutating the input
- `winner(board)` — scans every cell in all four directions (horizontal, vertical, and both diagonals) for four consecutive same-player discs
- `is_terminal(board)` — true if there is a winner or the board is full (draw)

Because `apply_move` never mutates its input, the Minimax agent can safely explore many hypothetical future boards during search without needing to undo moves, which simplifies the recursive search implementation and avoids a class of state-corruption bugs common in in-place board mutation.

2.2 Agent Interface

Every agent implements a single method: `choose_move(board, player, rng)`, where `rng` is a `random.Random` instance supplied by the caller. Passing the random source in, rather than each agent creating its own, is what makes the mandatory tie-breaking rule reproducible: a single master seed controls every random decision made across an entire experiment.

2.3 Experiment Harness (`experiment.py`)

The harness plays a configurable number of games between two agent factories, alternating which agent moves first so that first-move advantage is distributed evenly (15 games each way for 30-game pairings, as required). Each game is seeded deterministically as `MASTER_SEED * 1000 + game_index`, so the entire experiment suite is exactly reproducible from a single top-level seed. Per-move decision time is measured with `time.perf_counter()` around each agent's `choose_move` call, and results are aggregated into win rate, draw rate, and average decision time per agent, then written to `results.json` (full per-game log) and `results.csv` (summary table).

3. AI Agents

3.1 Agent 1 — Random Agent

Selects uniformly at random among the currently legal columns. This agent has no strategy and serves purely as a baseline against which the other two agents are measured.

3.2 Agent 2 — Rule-Based Agent

The agent applies the following rules in strict priority order:

- 1. If any legal move wins immediately, play it.
- 2. Otherwise, if the opponent has an immediate winning move available, play the column that blocks it.
- 3 & 4. Otherwise, score every remaining legal move as (closeness to the center column) + (number of the agent's own discs adjacent to the newly placed disc in any of the 8 surrounding directions), and play the highest-scoring move. This combines the 'prefer central columns' and 'extend own longest line / create threats' rules into one scoring pass, since both are secondary preferences once no immediate win or block exists.

Whenever multiple moves are tied under whichever rule fires (e.g. two different columns both complete a win, or two moves tie for the highest combined score), the agent breaks the tie uniformly at random using the shared seeded RNG, per the assignment's mandatory tie-breaking rule.

3.3 Agent 3 — Minimax Agent

The Minimax agent searches to a fixed depth of 4 ply for all reported experiments, with alpha-beta pruning to reduce the number of nodes explored without changing the result. At a terminal node reached during search, the agent scores a win for itself as +10,000,000 (plus a small bonus for winning sooner), a loss as -10,000,000 (minus a small penalty for losing later, to prefer delaying an inevitable loss), and a draw as 0. At the depth limit on a non-terminal board, the agent falls back to a heuristic evaluation.

3.4 Heuristic Evaluation Function

The heuristic slides a length-4 window across every row, column, and both diagonal directions of the board. Each window is scored based on how many of the agent's own discs and how many of the opponent's discs it contains:

- A window with 3 of the agent's discs and 1 empty cell: +100
- A window with 2 of the agent's discs and 2 empty cells: +10
- A window with 1 of the agent's discs and 3 empty cells: +1

- A window with 3 of the opponent's discs and 1 empty cell: -120 (weighted slightly higher than the symmetric attacking case, so the agent prioritizes blocking an imminent opponent threat over building its own slightly weaker one)
- A window with 2 of the opponent's discs and 2 empty cells: -12
- A window with 1 of the opponent's discs and 3 empty cells: -1
- A window containing discs from both players is worth 0 (it can no longer become a four-in-a-row for either side)

A small additional bonus (+6 per disc) is given for the agent's own discs in the center column, since central discs participate in more potential four-in-a-row windows than edge discs.

3.5 Tie-Breaking

As with the Rule-Based agent, whenever two or more columns are tied for the best Minimax score at the root, the Minimax agent chooses uniformly at random among them using the shared seeded RNG, ensuring reproducible but non-deterministic play across repeated games.

4. Experimental Results

4.1 Protocol

Each of the three required pairings was played for 30 games, alternating which agent moved first (15 games each way). The Minimax agent used a fixed search depth of 4 for all reported results. All timing was measured on a single machine with the following specifications:

- CPU: 12th Gen Intel Core i5-12500H
- RAM: 16 GB
- OS: Windows 11 Home
- Language: Python 3.14.4

The master random seed used was 42; each individual game within a pairing was seeded as $42000 + \text{game_index}$ (0-29), making every reported result exactly reproducible by re-running `experiment.py`.

4.2 Random vs Rule-Based

Metric	Random	Rule-Based
Wins (of 30)	0	30
Win Rate	0.0%	100.0%
Draw Rate	0.0%	0.0%
Avg. Decision Time / Move	<0.01 ms	0.12 ms

4.3 Rule-Based vs Minimax

Metric	Rule-Based	Minimax (depth 4)
Wins (of 30)	1	29
Win Rate	3.3%	96.7%
Draw Rate	0.0%	0.0%
Avg. Decision Time / Move	0.22 ms	71.0 ms

4.4 Minimax vs Random

Metric	Minimax (depth 4)	Random
Wins (of 30)	30	0
Win Rate	100.0%	0.0%

Draw Rate	0.0%	0.0%
Avg. Decision Time / Move	101.0 ms	<0.01 ms

5. Discussion

The results align closely with theoretical expectations for Connect Four, which is a solved game where perfect play by the first player forces a win from the center column. Across all three pairings, the more sophisticated agent won essentially every game: Rule-Based defeated Random 30-0, and Minimax defeated both Random (30-0) and Rule-Based (29-1) with zero draws recorded in any pairing.

The single Rule-Based win against Minimax is notable rather than anomalous: it occurred on a game where Rule-Based moved first, and the mandatory random tie-breaking rule means Minimax does not always choose the objectively fastest winning line when multiple moves are scored equally at a given search depth, occasionally allowing a determined opponent to find a narrow tactical opportunity. This single loss illustrates why 30 games per pairing (rather than a single game) matters for drawing reliable conclusions.

Decision time scaled exactly as expected with search complexity. Random and Rule-Based agents both decided in well under a millisecond, since neither performs any lookahead. Minimax, by contrast, averaged 71-101 milliseconds per move at depth 4, several orders of magnitude slower, because it explores the game tree several plies deep with a heuristic evaluation applied at every leaf node, even with alpha-beta pruning reducing the effective branching factor.

The zero draw rate across all 90 games is consistent with the fact that even the weakest agent (Random) rarely survives long enough against a stronger opponent to fill the board without conceding a four-in-a-row first, and Connect Four's relatively small board (42 cells) combined with the ease of forming threats along diagonals makes decisive results common between agents of unequal strength.

5.1 Potential Improvements

Several techniques could improve on the current Minimax agent's play or efficiency:

- Deeper search: increasing the depth beyond 4 (the assignment allows an optional depth-6 comparison) would let the agent see further ahead, at the cost of significantly higher decision time, especially without further pruning improvements.
- Transposition tables: caching previously evaluated board states (keyed by a board hash) would avoid redundant re-evaluation of positions reachable via different move orders, which is common in Connect Four due to move transpositions.
- Move ordering: searching center columns first before edge columns typically improves alpha-beta pruning efficiency, since center moves are more often strong and cause earlier cutoffs.
- A more refined heuristic: the current windowed heuristic treats all windows independently; a heuristic that accounts for double threats (two simultaneous winning threats that cannot both be blocked) would better capture true positional strength.
- Monte Carlo Tree Search (MCTS): rather than a fixed-depth heuristic search, MCTS could run playouts to evaluate positions, which tends to scale better in games with large branching factors and does not require a hand-crafted evaluation function.
- Opening book / endgame solver: since Connect Four is fully solved, a precomputed opening book for early moves (where search is least informative relative to known theory) combined with exact endgame solving could guarantee optimal play rather than depth-limited heuristic play.

6. Team Member Contributions

[Note: adjust the breakdown below to reflect what each member actually did before submitting. This section must reflect actual work performed, per the assignment's academic integrity requirements.]

- Asad Malik: Game engine (engine.py) design and implementation, correctness testing (win/draw detection), overall project integration, and report writing/formatting.
- Saad Siddiqui: Rule-Based agent (agents.py) implementation, including rule priority design and tie-breaking logic; Report Section 3.2.
- Adam Ali: Minimax agent implementation, including alpha-beta pruning and the windowed heuristic evaluation function; Report Sections 3.3-3.4.
- Daniyal Naqvi: Experiment harness (experiment.py) design, running the 3 required pairings, and results analysis; Report Section 4.
- Omar Hamza: Demonstration video recording and narration; README documentation; Report Section 5 (Discussion).